Midterm Solution

Q1.

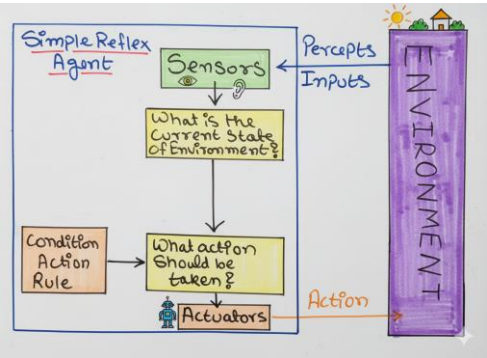
a. Artificial Intelligence (AI) means making computers or machines think and act like humans.
AI systems can:
- Think (analyze problems)
- Learn (from data or experience)
- Decide (choose the best action)
- Act (perform tasks such as speaking, driving, or recognizing faces)

The Turing Test checks if a computer can fool a human into thinking it's also human during a conversation.
If the human can't tell which is the computer, the machine passes the test.

b. An agent is anything that can perceive its environment and act on it.
Core Role: Bridge between environment and intelligent behaviour.
Environment → [Agent box] → Environment
The agent gets input (percepts) from its environment and takes actions that affect it.

Perception → Decision → Action → Environment
Example:
A cleaning robot sees dirt (perception), moves to it (action).

c. Simple Reflex Agent:

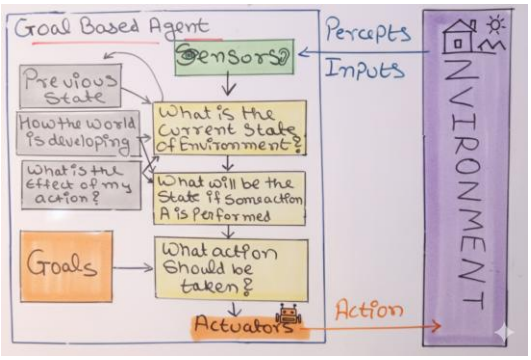


A Simple Reflex Agent is an AI agent that selects actions based only on the current percept using condition-action rules, without considering past percepts.
Example

Vacuum Cleaner Agent

- IF the current square is dirty → THEN clean it
- IF the current square is clean → THEN move to another square

Goal-Based Agent:



A Goal-Based Agent is an AI agent that selects actions by considering goals and choosing actions that help achieve the desired goal state.

Example

Navigation System

If the goal is **to reach the university**, the agent will:

- Analyze different routes
- Choose the path that leads to the destination

d. The **PEAS model** describes the task environment of an intelligent agent and consists of four components:
Performance Measure → evaluates the success of the agent.
Environment → the surroundings where the agent operates.
Actuators → devices used by the agent to perform actions.
Sensors → devices that help the agent perceive the environment.

Q2:

Uninformed Search

In Artificial Intelligence, Uninformed Search (Blind Search) is a search technique that does not use any additional knowledge about the goal. It explores the search space using only the information provided in the problem.

Examples: BFS, DFS, Uniform Cost Search.

Depth-First Search (DFS)

Depth-First Search (DFS) is an uninformed search algorithm that expands the deepest node in the search tree first before exploring other nodes.

Data Structure Used:
DFS uses a Stack (LIFO - Last In First Out) to manage its frontier.

Properties of DFS

1. Completeness
DFS is not complete in infinite search spaces because it may follow an infinite path. It is complete only if the search space is finite.

2. Optimality
DFS is not optimal. It does not guarantee the lowest-cost solution for both positive and negative step costs.

3. Time Complexity
Time complexity of DFS is:

$$O(b^m)$$

Where
b = branching factor
m = maximum depth of the search tree.

Step	**Current Node**	**Stack (Top at Right)**	**Visited Set**	**Action**
1	-	[A]	{A}	Start at **A**.
2	**A**	[C, B]	{A, B}	Pop **A**. Push neighbors **C** and **B**.

Step	Current Node	Stack (Top at Right)	Visited Set	Action
3	**B**	[C, E, D]	{A, B, D}	Pop **B**. Push neighbors **E** and **D**.
4	**D**	[C, E]	{A, B, D}	Pop **D**. (No unvisited neighbors).
5	**E**	[C]	{A, B, D, E}	Pop **E**. (No unvisited neighbors).
6	**C**	[G, F]	{A, B, D, E, C, F}	Pop **C**. Push neighbors **G** and **F**.
7	**F**	[G]	{A, B, D, E, C, F}	Pop **F**. (No unvisited neighbors).
8	**G**	[]	{A, B, D, E, C, F, G}	Pop **G**. **Goal Reached!**

DFS Traversal (Step by Step)

1. Start at **A**
2. Go to **B** (left child of A)
3. Go to **D** (left child of B)
4. Backtrack to **B**, then go to **E**
5. Backtrack to **A**, then go to **C**
6. Go to **F**
7. Go to **G**

DFS Order

A → B → D → E → C → F → G

Uniform-Cost Search (UCS)

In **Artificial Intelligence**, **Uniform-Cost Search (UCS)** is a search algorithm that **expands the node with the lowest path cost first**.

Working of UCS

1. Start from the **initial node**.
2. Add it to the **open list**.
3. Always select the node with the **lowest cumulative path cost (g(n))**.
4. Expand that node and add its children to the list with updated costs.
5. Continue until the **goal node is reached**.

Property	**UCS Result**
Data Structure	Priority Queue
Completeness	Complete for positive costs
Optimality	Optimal for positive costs
Time Complexity	O(b^(C*/ε))

Step	**Node Expanded**	**Priority Queue (Node, Cost)**	**Visited / Explored**	**Action**
1	—	[(S, 0)]	{S}	Start at **S** with cost 0.
2	**S**	[(A, 1), (B, 4)]	{S, A, B}	Expand **S**. Add **A** (0+1) and **B** (0+4).

Step	Node Expanded	Priority Queue (Node, Cost)	Visited / Explored	Action
3	**A**	[(C, 4), (B, 4), (D, 6)]	{S, A, B, C, D}	Expand **A** (cost 1). Add **C** (1+3) and **D** (1+5).
4	**C**	[(B, 4), (D, 6), (E, 9)]	{S, A, B, C, D, E}	Expand **C** (cost 4). Add **E** (4+5).
5	**B**	[(M, 5), (D, 6), (E, 9)]	{S, A, B, C, D, E, M}	Expand **B** (cost 4). Add **M** (4+1).
6	**M**	[(D, 6), (E, 9)]	{S, A, B, C, D, E, M}	Expand **M** (cost 5). (No neighbors).
7	**D**	[(G, 8), (E, 9), (F, 10)]	{S, A, B, C, D, E, M, G, F}	Expand **D** (cost 6). Add **G** (6+2) and **F** (6+4).
8	**G**	[(E, 9), (F, 10)]	{S, A, B, C, D, E, M, G, F}	Expand **G** (cost 8). **Goal Reached!**